Solution: Word Ladder II

Problem Statement

A `transformation sequence` is a sequence that starts from the word `beginWord`, ends with `endWord`, and contains a word from the given dictionary `wordList` in the middle i.e. a sequence of words `beginWord -> s1 -> s2 -> ... -> sk` such that:

- Each adjacent pair of words differs by a single letter.
- Each s_i for `1 <= i <= k` is in wordList. sk == endWord
- The `beginWord` does not need to be in wordList.

Given two strings, `beginWord` and `endWord`, and a `wordList`, return all the `shortest transformation sequences`, or an empty list if no such sequence exists. Each sequence should be in the form of `[beginWord, s1, s2, ..., sk]`.

Examples

1. **Example 1**
   - **Input**: start = "cat", end = "dog", wordList = ["dat", "dot", "dog"]
   - **Expected Output**: [["cat", "dat", "dot", "dog"]]
   - **Justification:** The sequence "cat" -> "dat" -> "dot" -> "dog" changes one letter at a time, and each word exists in the wordList. It's the shortest path from "cat" to "dog".
2. **Example 2**
   - **Input**: start = "bit", end = "fog", wordList = ["but", "put", "big", "pot", "pog", "dog"]
   - **Expected Output**: []
   - **Justification**: Although there are words in the list that are one letter away from "bit", there's no sequence that successfully transforms "bit" to "fog" through words in the wordList.
3. **Example 3**
   - **Input**: start = "hot", end = "cog", wordList = ["dot", "dog", "lot", "log", "cog"]
   - **Expected Output**: [["hot", "dot", "dog", "cog"], ["hot", "lot", "log", "cog"]]
   - **Justification**: This example shows two possible shortest transformation sequences from "hot" to "cog". Each sequence involves changing one letter at a time with all intermediate words existing in the given wordList.

Solution

To solve this problem, we'll use a breadth-first search (BFS) strategy to explore all possible word transformations, ensuring that we only proceed through valid words in the given dictionary. This approach is effective because it naturally finds the shortest paths first, avoiding unnecessary exploration of longer paths. We'll maintain a queue to keep track of the paths explored at each step and a set to record words that have already been visited to prevent revisiting them. By exploring words in layers and tracking the path taken to reach each word, we can ensure that once we reach the target word, all paths found are of the shortest possible length.

The key to efficiency in this approach is to dynamically generate possible word transformations by changing each letter of the current word to every letter from 'a' to 'z' and checking if the new word exists in the wordList. This method minimizes the number of unnecessary comparisons and quickly identifies viable paths. The use of a queue facilitates the BFS approach, and tracking visited words ensures we don't cycle through the same words repeatedly.

Step-by-step Algorithm

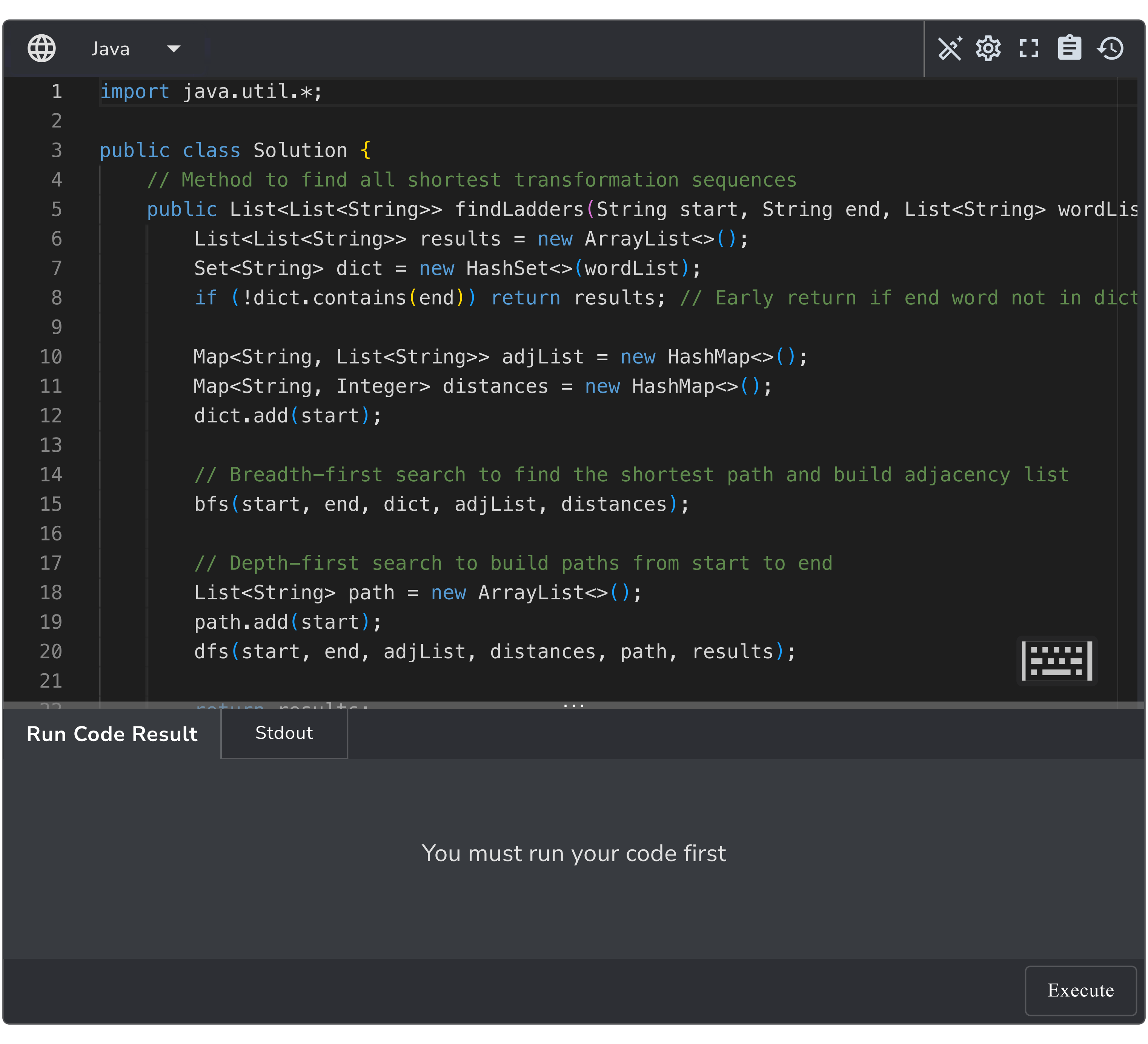
1. **Initialize Data Structures**:
   - **Word List:** Convert the input word list into a set for efficient lookup.
   - **Adjacency List:** Create a map (or dictionary) to store the adjacency list, representing the graph of word transformations.
   - **Distances:** Create a map (or dictionary) to store the distance (number of transformations) from the start word to each word encountered during BFS.
2. **Check for End Word**:
   - Check if the `endWord` exists in the `wordList`. If it does not, return empty list immediately since transformation is impossible without the `endWord` being in the list.
3. **Breadth-First Search (BFS)**:
   - Initialize a queue and add the start word to it. Set the distance for the start word to 0 in the distances map.
   - While the queue is not empty, process each word in the queue:
     - For the current word, generate all possible one-letter transformations.
     - For each transformation that is in the word list and has not been visited (i.e., not in the distances map):
       - Add it to the queue.
       - Set its distance as one more than the current word's distance.
       - Add the transformation as an adjacent word to the current word in the adjacency list.
   - Continue until the queue is empty or the end word is reached.
4. **Depth-First Search (DFS)**:
   - Start DFS from the start word with the current path containing only the start word.
   - For each word in the current path, explore all adjacent words (transformations) as per the adjacency list.
   - If an adjacent word is the next step towards the end (i.e., its distance is one more than the current word's distance), add it to the current path and continue DFS with this new word.
   - If the end word is reached, add the current path to the results.
   - Backtrack by removing the last word from the current path before returning to explore another branch.
5. **Returning Results:**
   - After completing DFS, return all found paths that transform the start word to the end word through valid transformations in the word list.

Algorithm Walkthrough

Let's consider the input: start = "hot", end = "cog", wordList = ["dot", "dog", "lot", "log", "cog"]

1. **Initialization:**
   - The word list is converted to a set for efficient lookups: `{"dot", "dog", "lot", "log", "cog"}`.
   - Adjacency list and distances are initialized as empty structures.
2. **BFS to Build Adjacency List and Distances:**
   - Enqueue "hot" and set its distance to 0.
   - Dequeue "hot" and explore its neighbors (one-letter transformations present in the word list): "dot" and "lot".
   - Enqueue "dot" and "lot", setting their distances to 1. Update the adjacency list to show "hot" is connected to "dot" and "lot".
   - Continue BFS:
     - Dequeue "dot", find its neighbors: "dog". Enqueue "dog", set its distance to 2, and update the adjacency list.
     - Dequeue "lot", find its neighbors: "log". Enqueue "log", set its distance to 2, and update the adjacency list.
     - Dequeue "dog", find its neighbor: "cog". Enqueue "cog", set its distance to 3, and update the adjacency list.
     - Dequeue "log", which also leads to "cog", but as "cog" is already visited with the same distance, just update the adjacency list.
   - BFS ends when the queue is empty.
3. **DFS to Find All Paths:**
   - Start DFS with "hot". The current path is ["hot"].
   - Explore paths from "hot" to its neighbors: "dot" and "lot".
     - For "dot", the path becomes ["hot", "dot"]. Continue DFS from "dot":
       - Only neighbor "dog" has the correct distance. Path becomes ["hot", "dot", "dog"].
       - From "dog", go to "cog". Path is ["hot", "dot", "dog", "cog"]. This is a valid path, add it to results.
     - Backtrack to ["hot"] and then to ["hot", "lot"].
       - From "lot", go to "log". Path becomes ["hot", "lot", "log"].
       - From "log", go to "cog". Path is ["hot", "lot", "log", "cog"]. This is another valid path, add it to results.
   - DFS completes once all paths from "hot" have been explored.
4. **Results:**
   - The algorithm finds two shortest transformation sequences: `["hot", "dot", "dog", "cog"]` and `["hot", "lot", "log", "cog"]`.

Code



```java
import java.util.*;

public class Solution {
    // Method to find all shortest transformation sequences
    public List<List<String>> findLadders(String start, String end, List<String> wordLis
        List<List<String>> results = new ArrayList<>();
        Set<String> dict = new HashSet<>(wordList);
        if (!dict.contains(end)) return results; // Early return if end word not in dict

        Map<String, List<String>> adjList = new HashMap<>();
        Map<String, Integer> distances = new HashMap<>();
        dict.add(start);

        // Breadth-first search to find the shortest path and build adjacency list
        bfs(start, end, dict, adjList, distances);

        // Depth-first search to build paths from start to end
        List<String> path = new ArrayList<>();
        path.add(start);
        dfs(start, end, adjList, distances, path, results);

```

You must run your code first

Complexity Analysis

Time Complexity: $O(NK^2 + \alpha)$

- **BFS Complexity**: The BFS part of the algorithm involves traversing each word in the `wordList` (N words) and for each word, generating all possible one-letter transformations (at most K^2 transformations, where K is the length of the word). This results in a time complexity of $O(NK^2)$ for the BFS portion.
- **Backtracking Complexity:** The backtracking (DFS) part is used to find all shortest paths from `start` to `end`, which has a time complexity of $O(\alpha)$, where α represents the number of such paths.
- **Overall Time Complexity**: The overall time complexity of the algorithm is $O(NK^2 + \alpha)$, accounting for both the BFS construction of the adjacency list and distances, as well as the DFS to enumerate all shortest paths.

Space Complexity: $O(NK)$

- **Storage of `wordList`**: Copying the `wordList` into a set or similar data structure for efficient lookups requires $O(NK)$ space, where N is the number of words, and K is the maximum length of a word.
- **Adjacency List:** The BFS algorithm constructs an adjacency list to represent the graph, which requires additional space. However, the space required for storing the adjacency list and distances map is dominated by the space needed to store the words themselves, contributing to the overall space complexity of $O(NK)$.
- **Overall Space Complexity**: Hence, the total space complexity of the algorithm is $O(NK)$, dominated by the storage of the `wordList` and the auxiliary data structures (adjacency list, distances map, and the recursion stack during DFS).

← Previous
Word Ladder II (Hard)

Next →
Number Of Flowers In Full Bloom (Hard)

☐ Mark as Completed